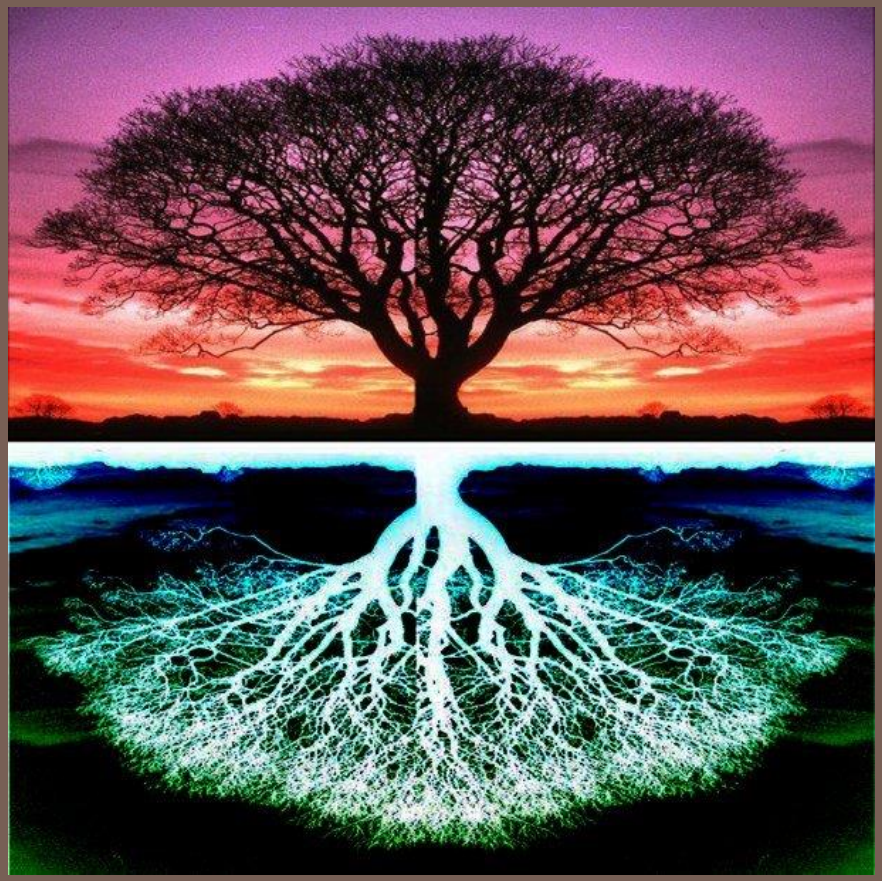


TREES

assadullahkhan

AssadUllahKhan

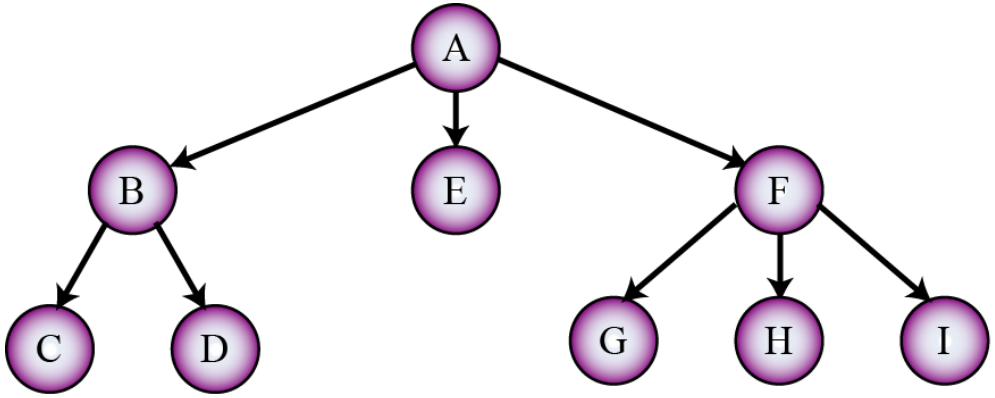
Research Assistant: **Mr. Assad Ullah Khan**

Outline

- Introduction to Trees
- Tree Terminologies
- Types of Tree
- Application of Trees
- Tree Traversals
- Binary Tree
- Expression Tree
- Binary Search Tree

TREES

A **tree** consists of a finite set of elements, called **nodes** (or **vertices**) and a finite set of directed **lines**, called **arcs**, that connect pairs of the nodes.



A: root
B and F: internal nodes
C, D, E, G, H, and I: leaves

Nodes

Figure **Tree representation**

TREES

We can divided the vertices in a tree into three categories: the **root, leaves** and the **internal nodes**. Table 12.1 shows the number of **outgoing** and **incoming** arcs allowed for each type of node.

Table 12.1 Number of incoming and outgoing arcs

Type of node	*Incoming arc*	*Outgoing arc*
root	0	0 or more
leaf	1	0
internal	1	1 or more

SUBTREES

Each node in a tree may have a subtree. The subtree of each node includes one of its children and all descendents of that child. Following Figure shows all subtrees for the tree in Previous Figures

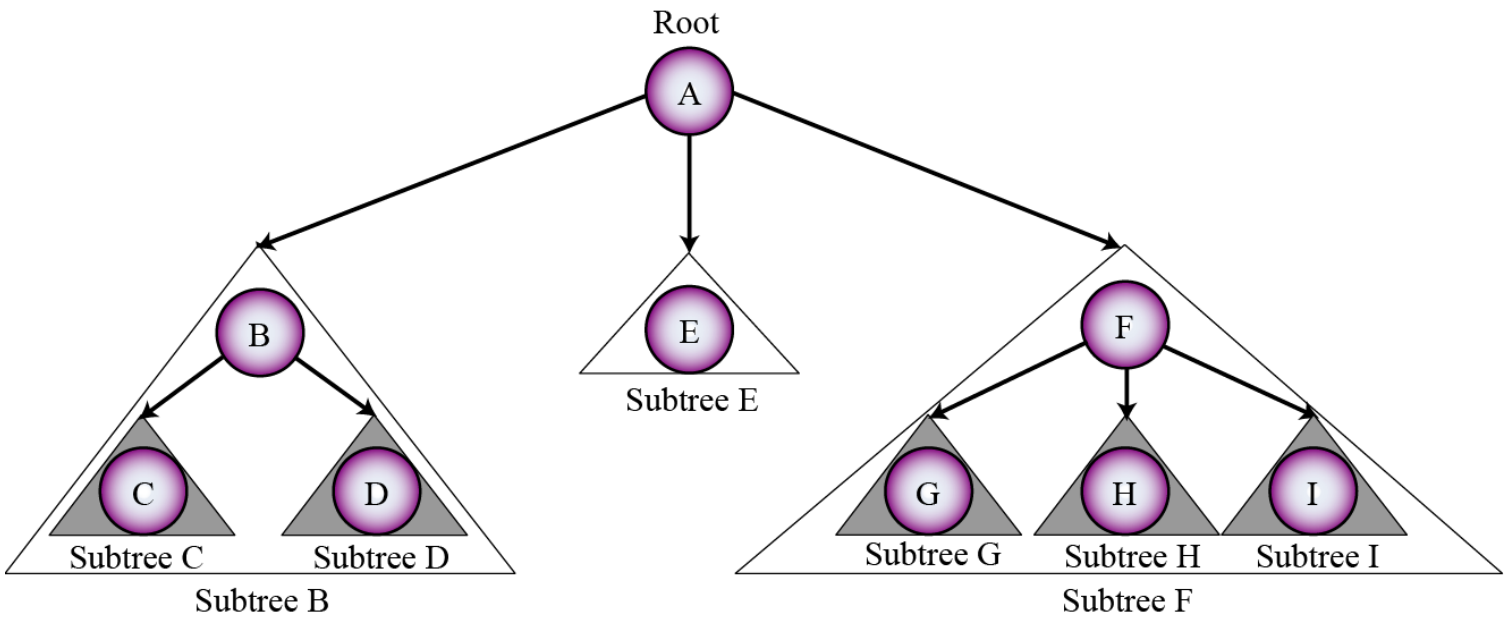


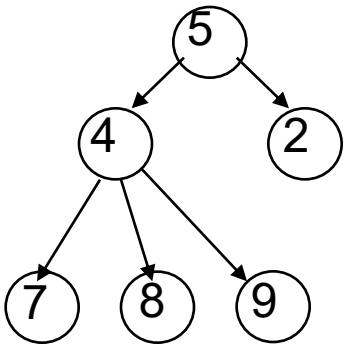
Figure **Subtrees**

Tree Overview

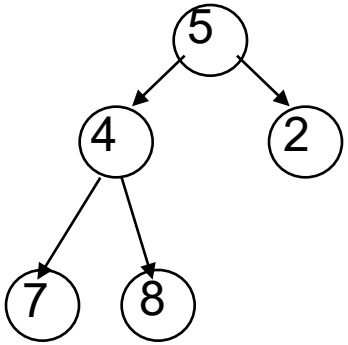
Tree: recursive data structure (similar to linked-list)

- Each node may have zero or more *successors* (children)
- Each node has exactly one *predecessor* (parent) except the *root*, which has none
- All nodes are reachable from *root*

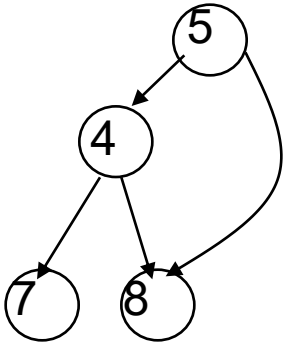
Binary tree: tree in which each node can have at most two children: a left child and a right child



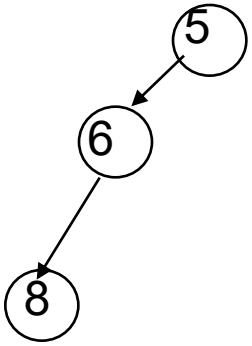
General tree



Binary tree



Not a tree



List-like tree

Tree Terminologies

- ***Node:*** Each element is called a Node
- ***Root Node:*** The node from which all other nodes originate and it has no parent is called Root Node
- ***Parent Node:*** The node from which other nodes arises
- ***Child Node:*** The node that is originated from other node arise
- ***Leaf Node:*** The node from which no other nodes arises
- ***Siblings:*** The children of the same parents are called siblings
- ***Edge:*** The line that connects two nodes is called Edge
- ***Path:*** Combining a number of edges forms a path
- ***Level:*** In a tree data structure, root node is said to be 0, and the children of root node are at Level 1and so on

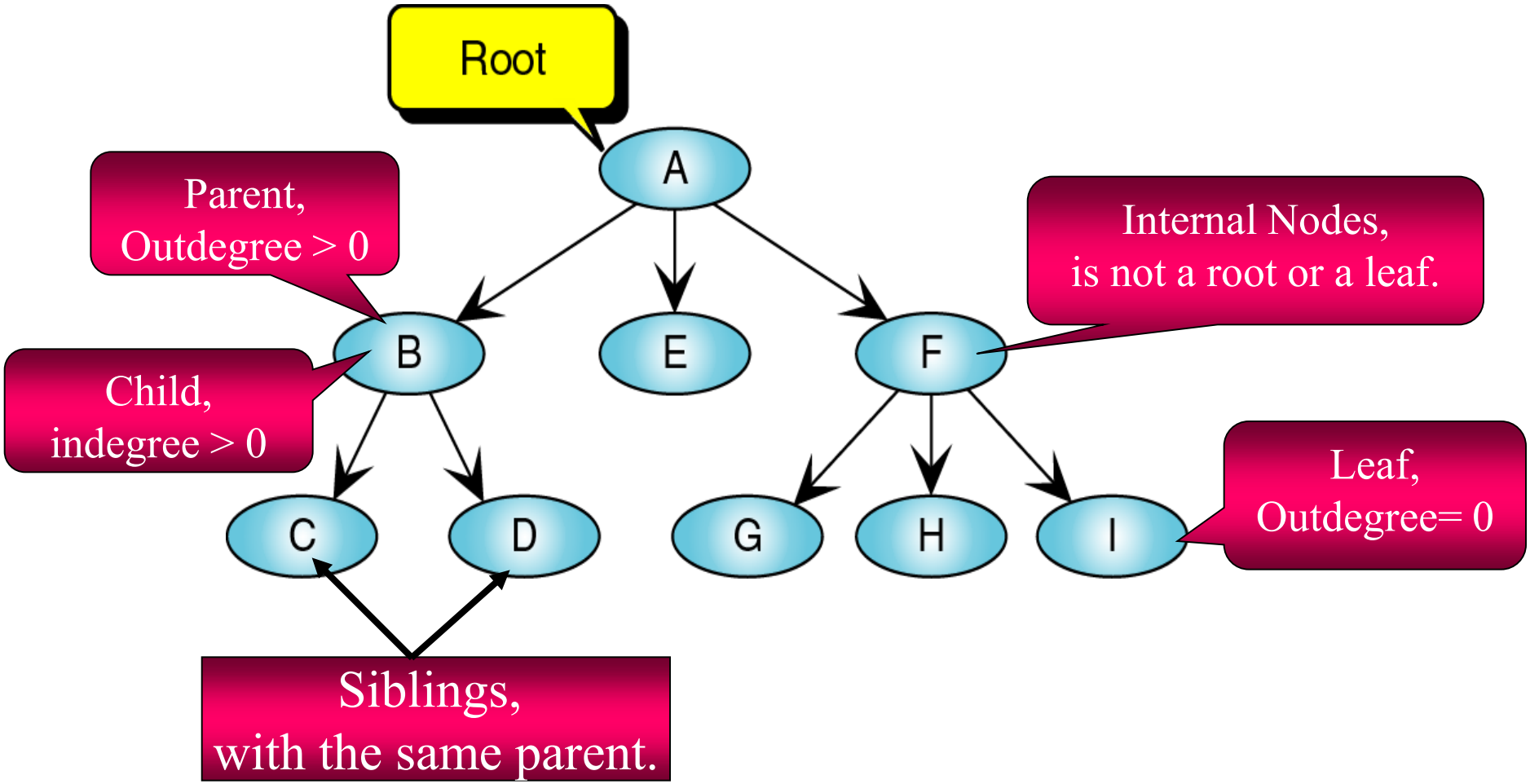
Tree Terminologies

- ***Length:*** Number of Edges on the path
- ***Depth of a Node:***
  - length of the unique path from the root to that node
  - root is at depth 0
- ***Depth of Tree:***
  - It is equal to the depth of the deepest leaf from root Node
- ***Height of a Node:***
  - length of the longest path from that node to a leaf,
  - all leaves are at height 0
- ***Height of Tree:***
  - It is equal to the depth of the deepest leaf from root Node

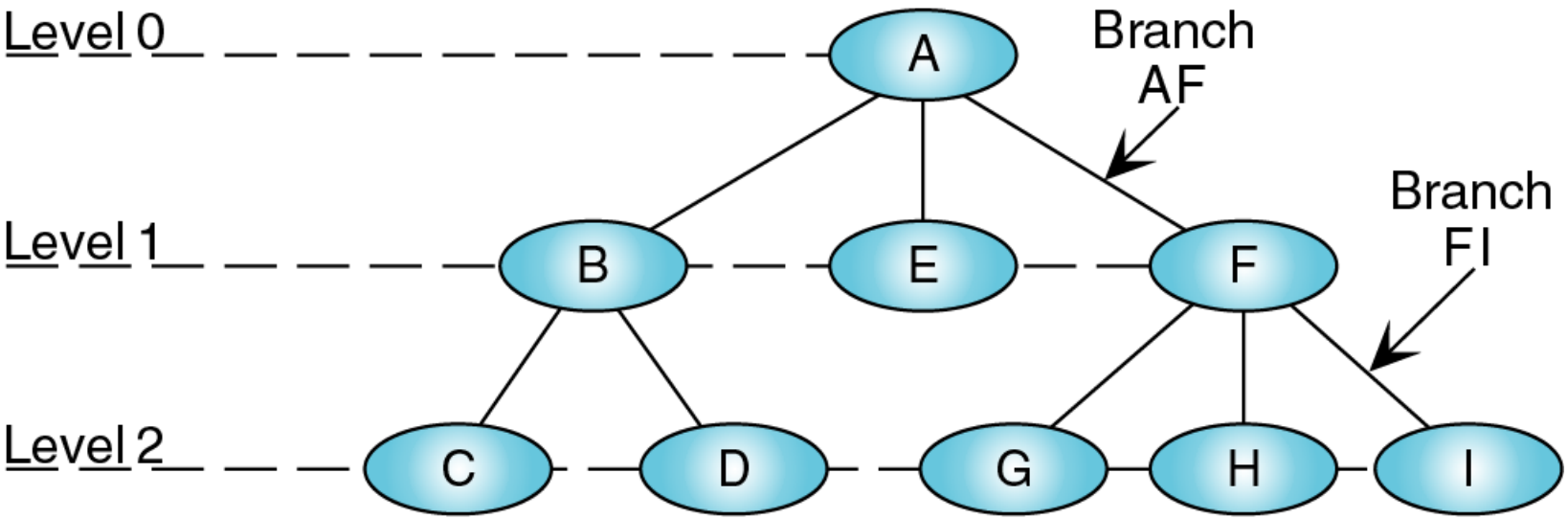
Tree Terminologies

- ***Branch:*** A path ending is a leaf is called a branch
- ***Degree of a Node:*** It's the total number of children of a node
- ***Internal Node:*** It's neither leaf nor root
- ***Forest:*** It's collection of trees but is not necessarily connected
- ***Descendant:***
  - A node reachable by repeated proceeding from parent to child.
- ***Ancestor:***
  - A node reachable by repeated proceeding from child to parent.
- **Grandparent** and **Grandchild** relations can also be defined in a similar manner.

Basic Concepts



Basic Concepts



Parents:	A, B, F	Leaves	C,D,E,G,H,I
Children:	B, E, F, C, D, G, H, I	Internal nodes	B,F
Siblings:	{B,E,F},{C,D}, {G,H,I}		

Height of tree = max. Level of leaf + 1

Tree terminology

M: root of this tree

G: *root* of the *left subtree* of M

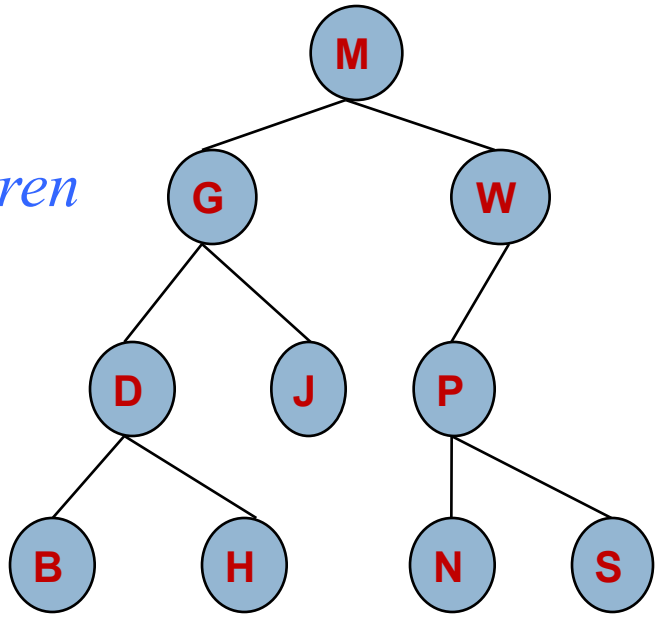
B, H, J, N, S: *leaves (their set of children is empty)*

N: *left child* of P; S: right *child* of P

P: parent of N

M and G: *ancestors* of D

P, N, S: *descendents* of W



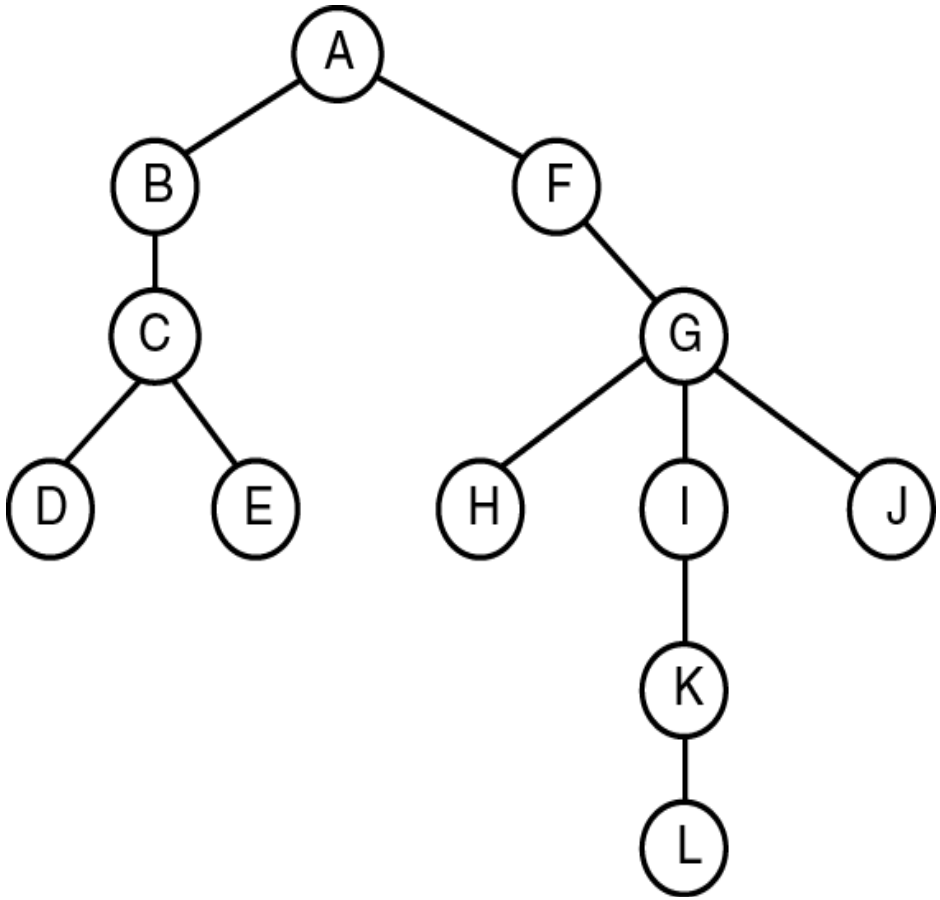
J is at *depth* 2 (i.e. length of path from root = no. of edges)

W is at *height* 2 (i.e. length of <u>longest</u> path to a leaf)

A collection of several trees is called a ...?

Exercise (Find the given)

1. Root and Leaves
2. Internal nodes
3. Height of a Tree
4. Depth of a Tree
5. Ancestors of H
6. Descendants of F
7. Degree of A
8. Indegree of F
9. Outdegree of B
10. Level of G
11. Height of node I
12. Depth of node G
13. Identify Siblings
14. Path from C to J



Application of Trees

- Trees are used in computer science;
  - To represent algebraic formulas,
  - As an efficient method for searching large dynamic lists,
  - For such diverse applications as artificial intelligence systems,
  - And encoding algorithms.

Tree traversals

"Walking" over whole tree is a tree traversal

- Done often enough that there are standard names

Previous example: inorder traversal

- Process left subtree
- Process root
- Process right subtree

Note: Can do other processing besides printing

Other standard kinds of traversals

- preorder traversal
  - Process root
  - Process left subtree
  - Process right subtree
- postorder traversal
  - Process left subtree
  - Process right subtree
  - Process root
- level-order traversal
  - Not recursive uses a queue.

BINARY TREES

A binary tree is a tree in which no node can have more than two subtrees. In other words, a node can have zero, one or two subtrees.

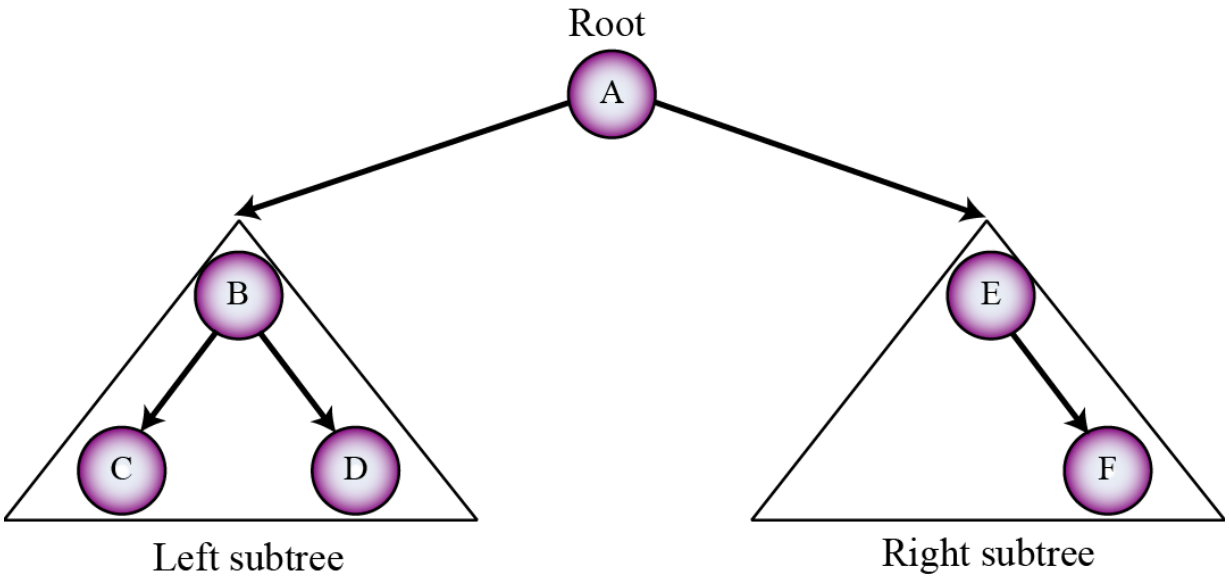
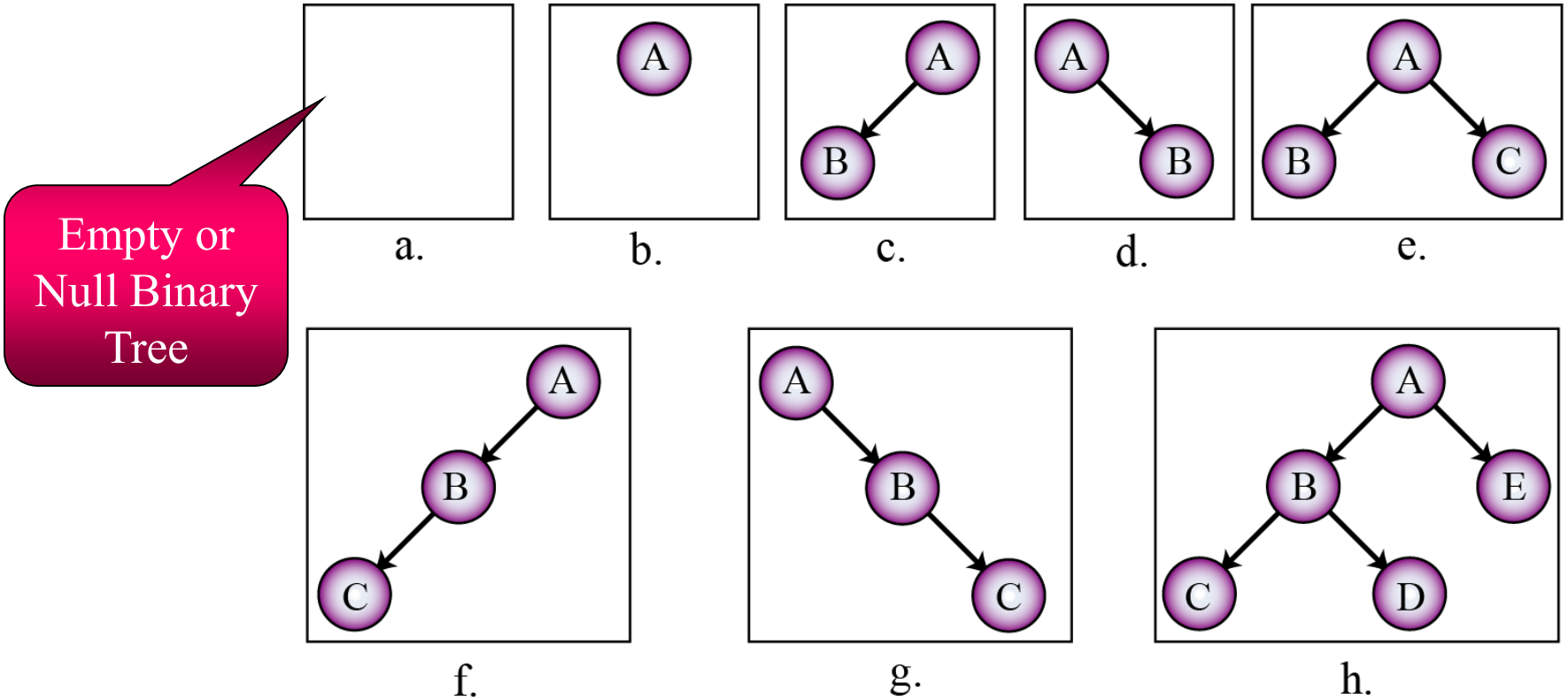


Figure A binary tree

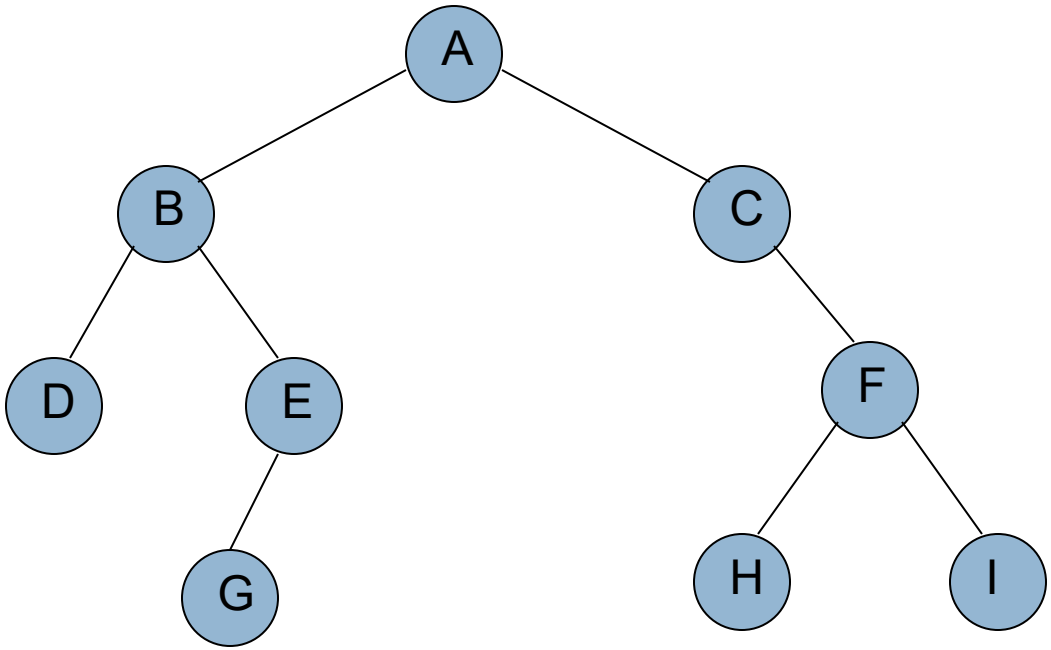
BINARY TREES

A binary tree is either empty or consist of a node, root, with two subtrees, in which each subtree is also a binary tree
The following Figure shows eight trees,

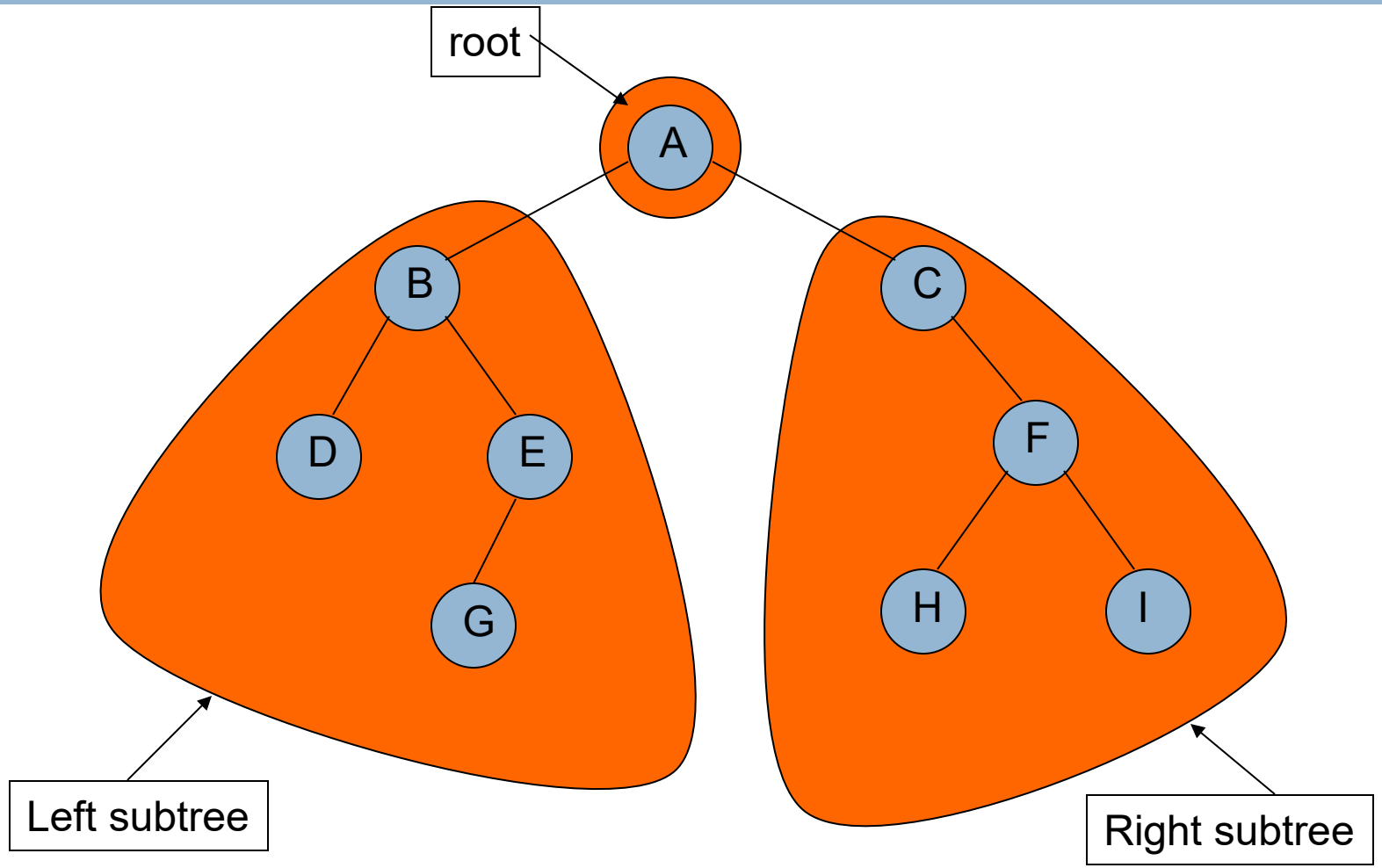


Binary Tree

- Binary tree with 9 nodes.

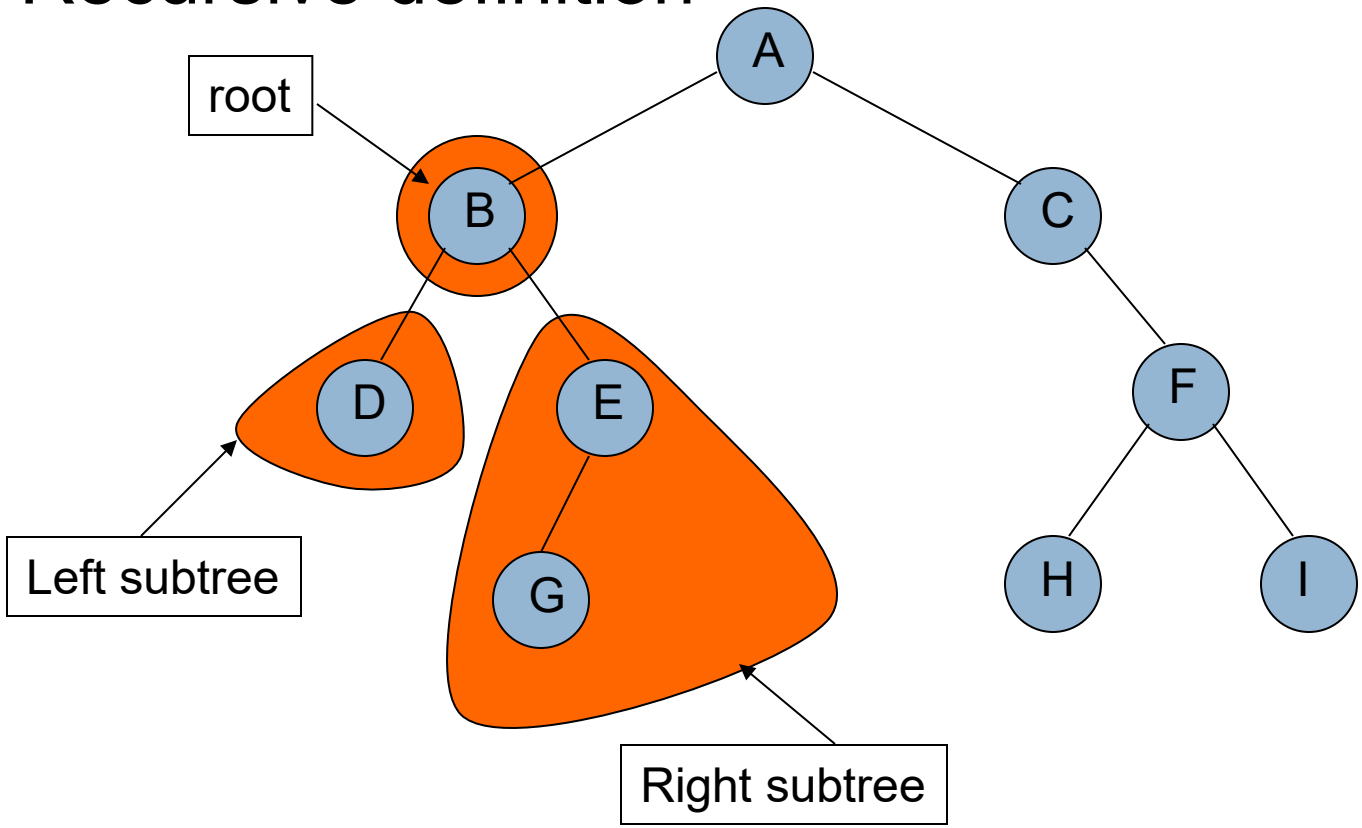


Binary Tree



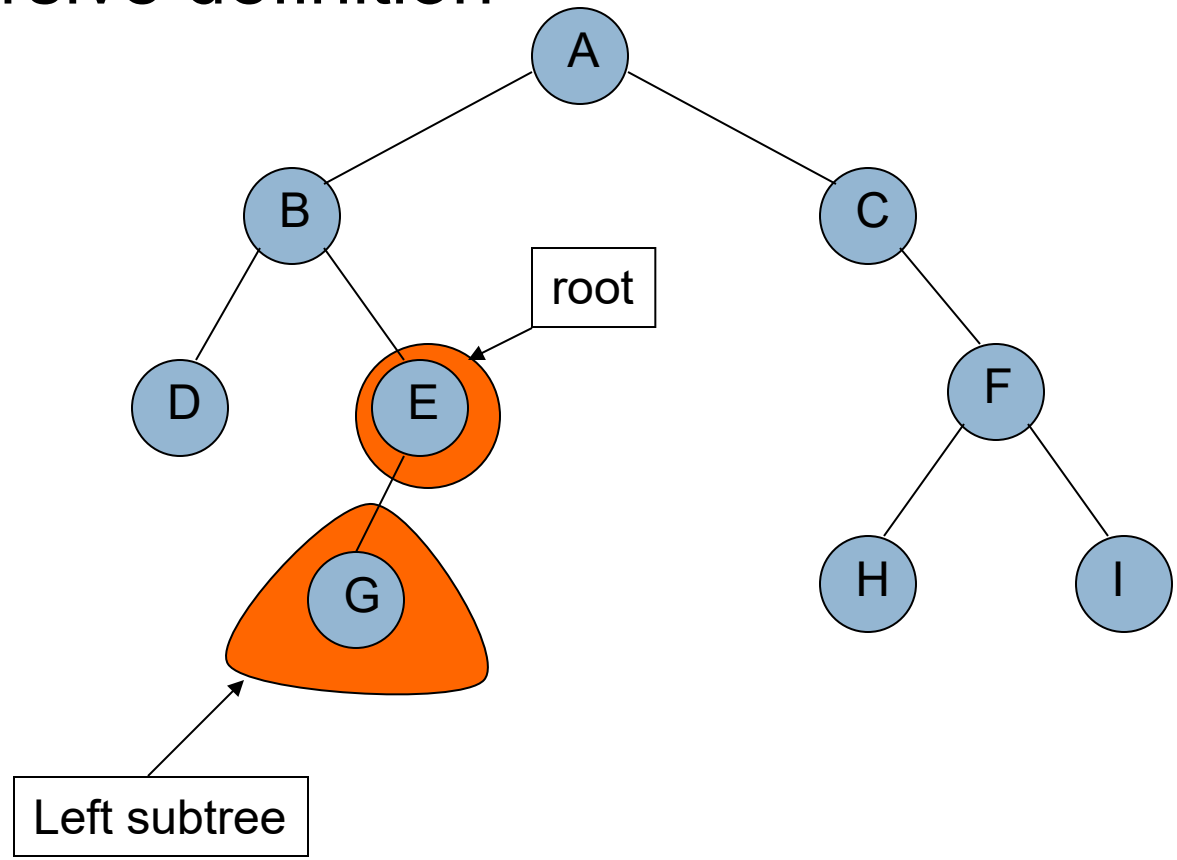
Binary Tree

- Recursive definition



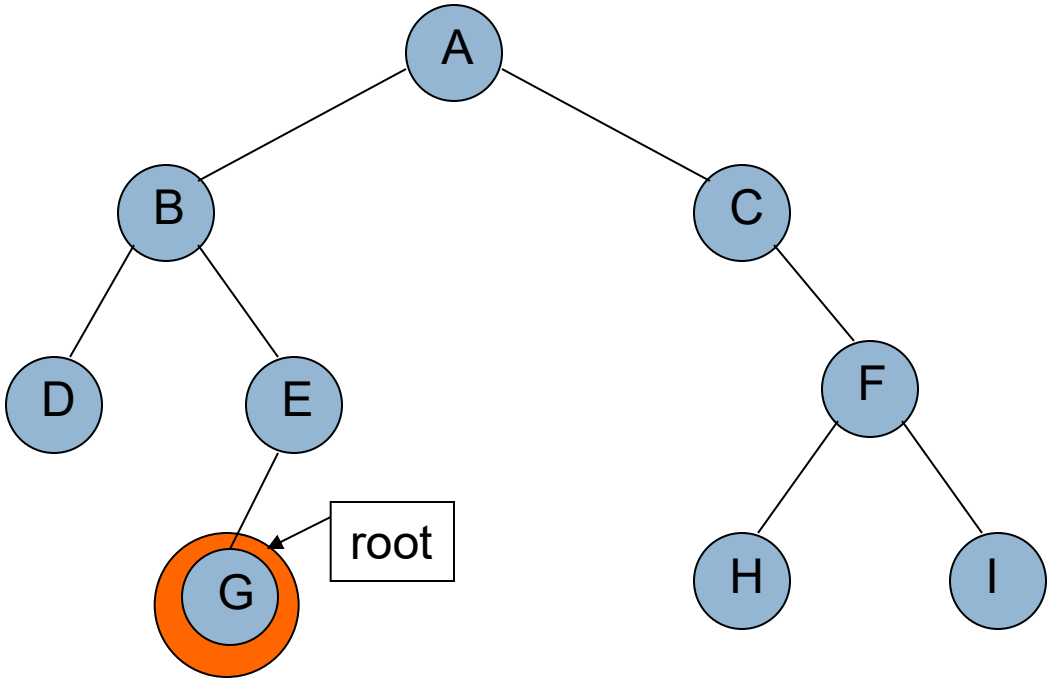
Binary Tree

- Recursive definition



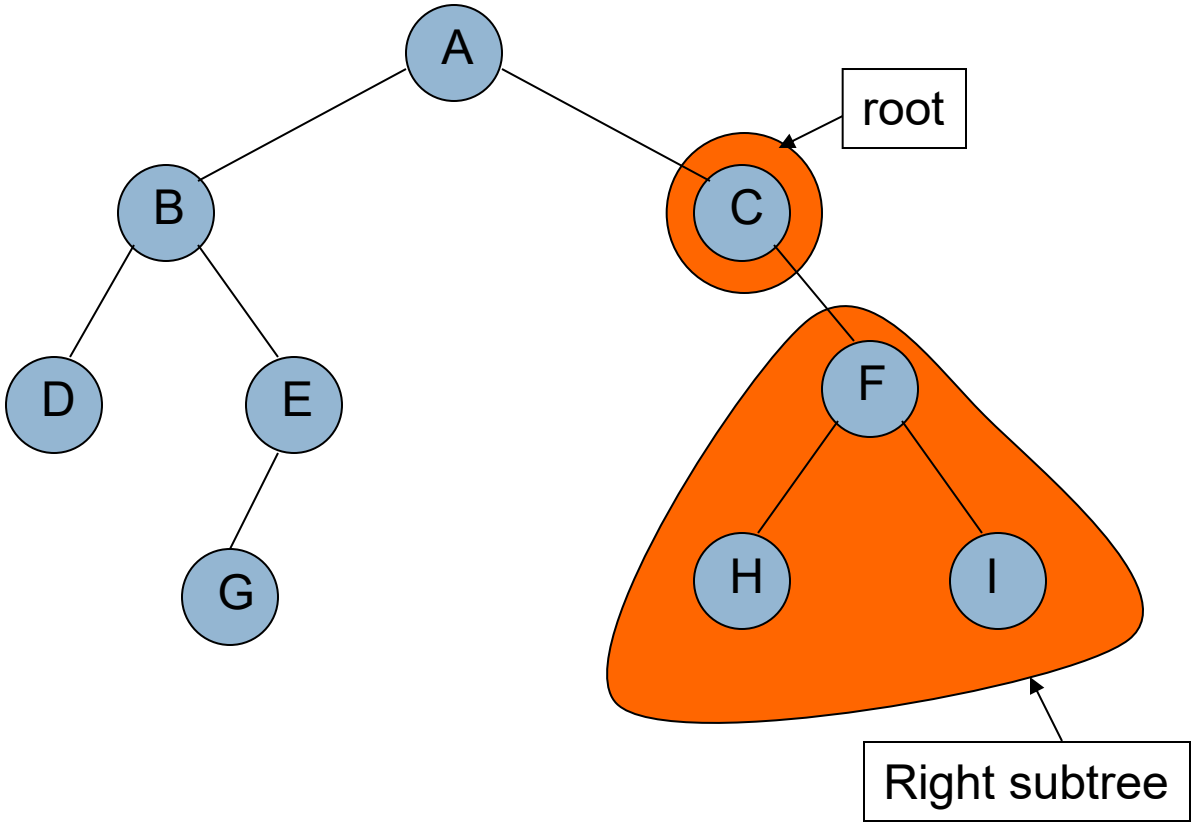
Binary Tree

- Recursive definition



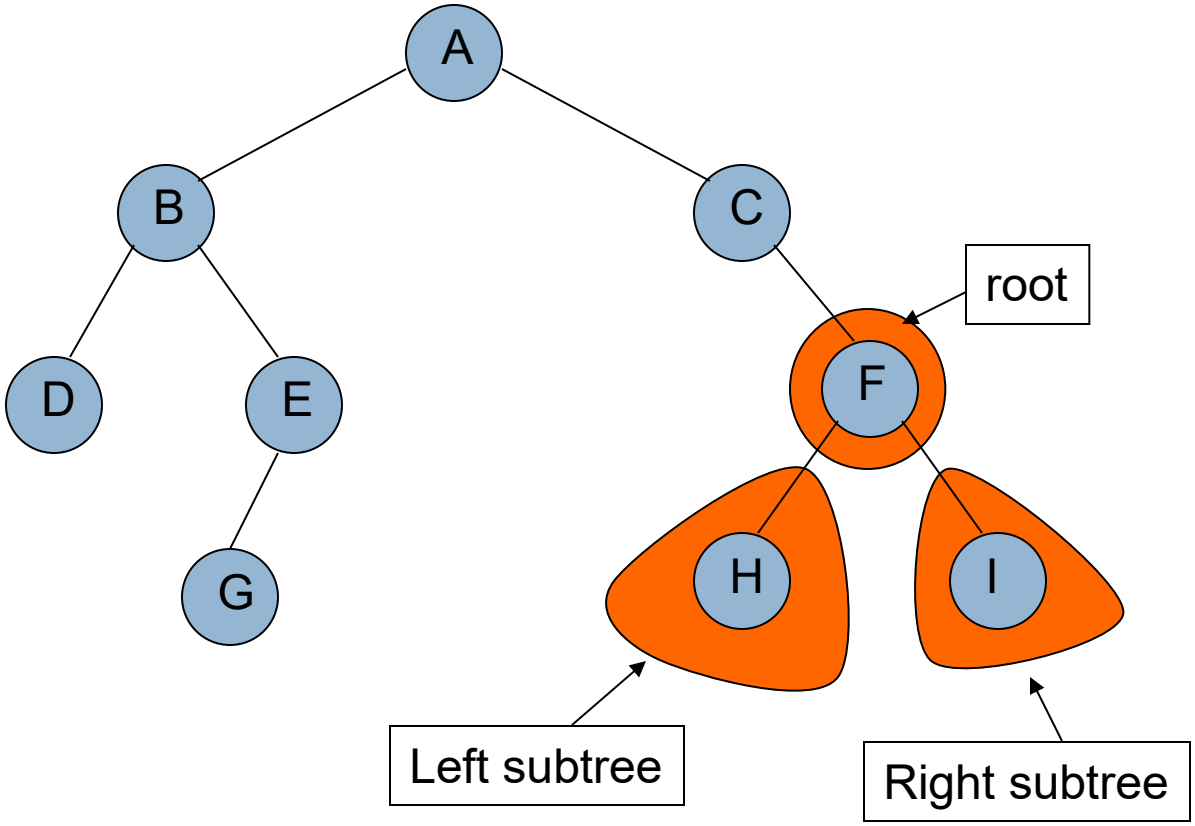
Binary Tree

- Recursive definition



Binary Tree

- Recursive definition



Binary Tree Traversals

A binary tree traversal requires that each node of the tree be processed once and only once in a predetermined sequence. The two general approaches to the traversal sequence are depth-first and breadth-first traversal.

Visiting − Visiting refers to checking the value of a node when control is on the node.

Traversing − Traversing means passing through nodes in a specific order.

Binary Tree Traversals

- **Traversing**

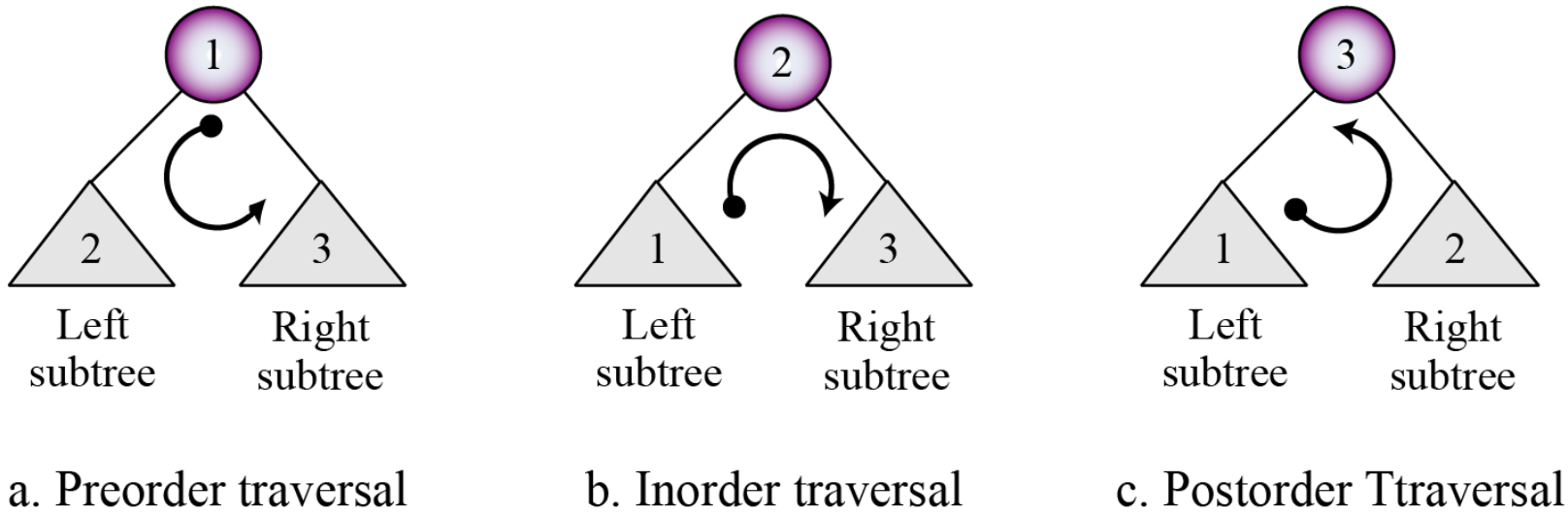


Figure Depth-first traversal of a binary tree

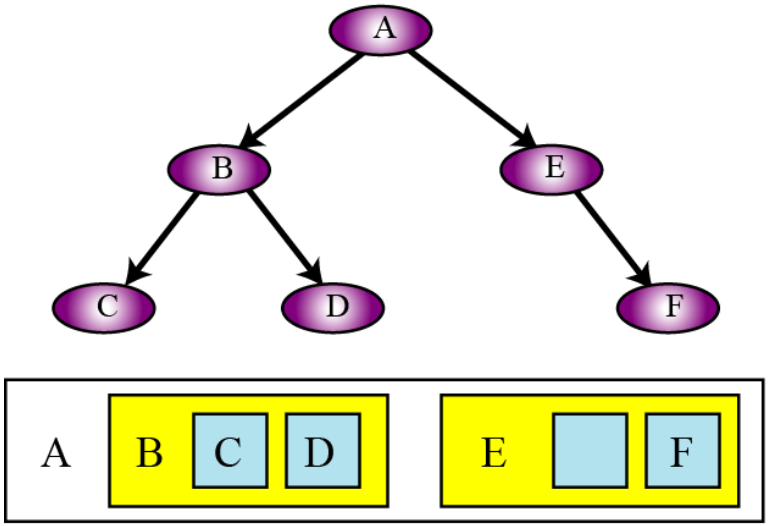
Types of Binary Tree Traversals

There are two broad Categories of Traversal

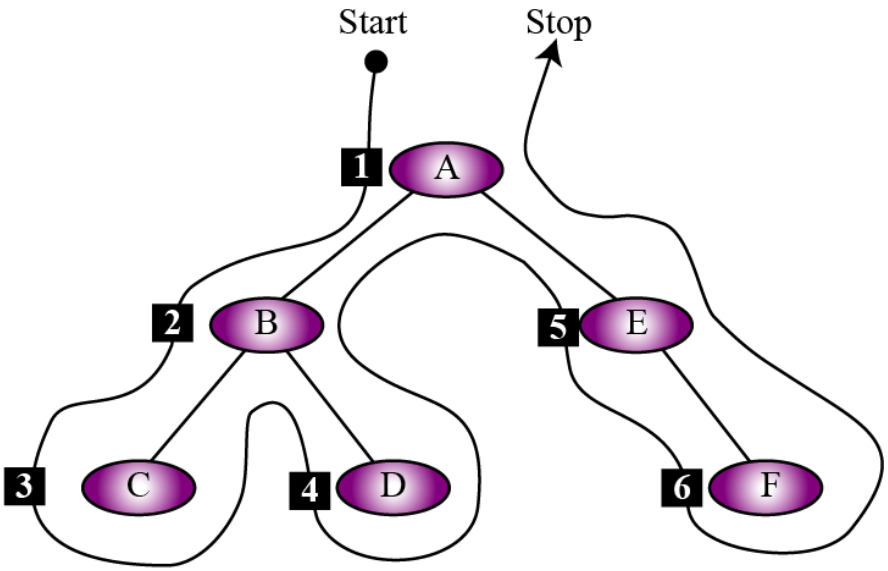
- Depth First Traversal
  - Preorder Traversal
    - vertex, left subtree, right subtree (Parent, Left, Right)
  - Inorder traversal
    - left subtree, vertex, right subtree (Left, Parent, Right)
  - Postorder traversal
    - left subtree, right subtree, vertex (Left, Right, Parent)
- Breadth First Traversal
  - Level order traversal
    - vertex, left children, right children

Depth First Traversal

- The following Figure shows how we visit each node in a tree using **preorder traversal**. The figure also shows the walking order. In **preorder traversal** we visit a node when we pass from its left side. The nodes are visited in this order: A, B, C, D, E, F.



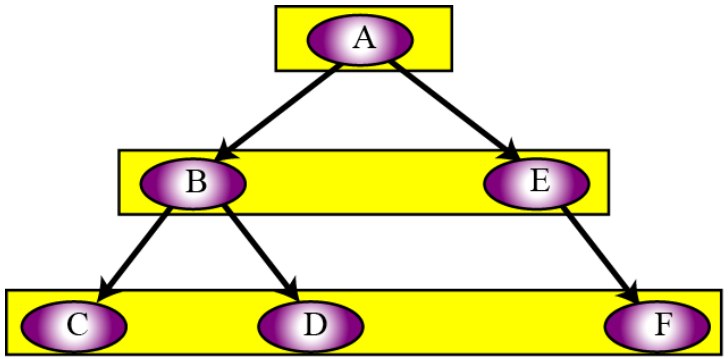
a. Processing order

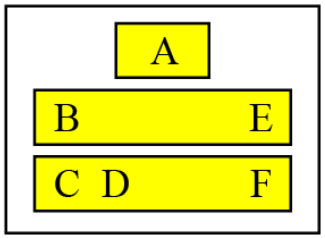


b. "Walking" order

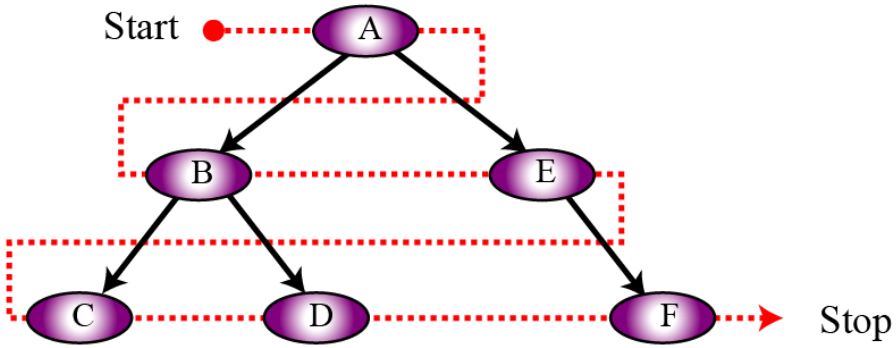
Breadth First Traversal

The following Figure shows how we visit each node in a tree using **breadth-first traversal (Level by Level Traversal.** The figure also shows the walking order. The traversal order is A, B, E, C, D, F.



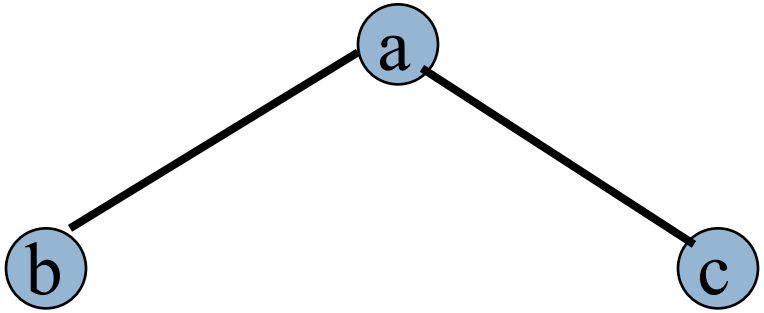


a. Processing order



b. "Walking" order

Preorder Example (Visit = print)

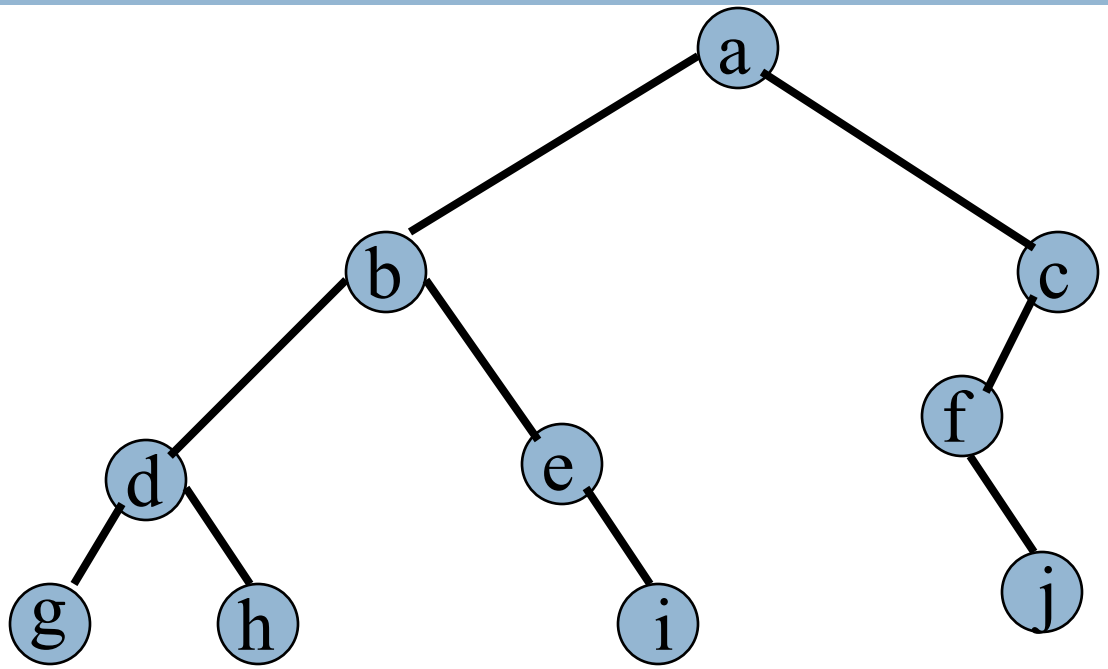


a b c

Algorithm

- Until all nodes are traversed −
  - **Step 1** − Visit root node.
  - **Step 2** − Recursively traverse left subtree.
  - **Step 3** − Recursively traverse right subtree.

Preorder Example (Visit = print)

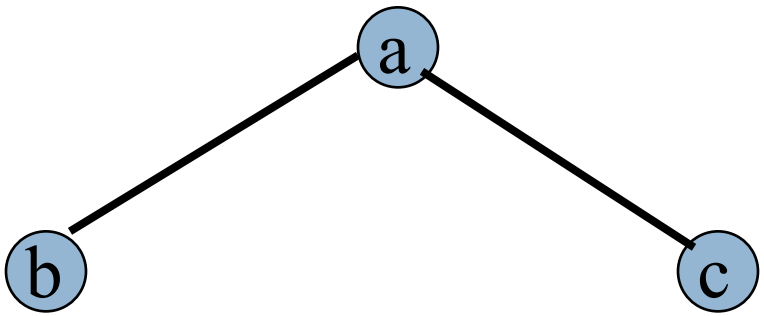


a b d g h e i c f j

Algorithm

- Until all nodes are traversed −
  - **Step 1** − Visit root node.
  - **Step 2** − Recursively traverse left subtree.
  - **Step 3** − Recursively traverse right subtree.

Inorder Example (Visit = print)

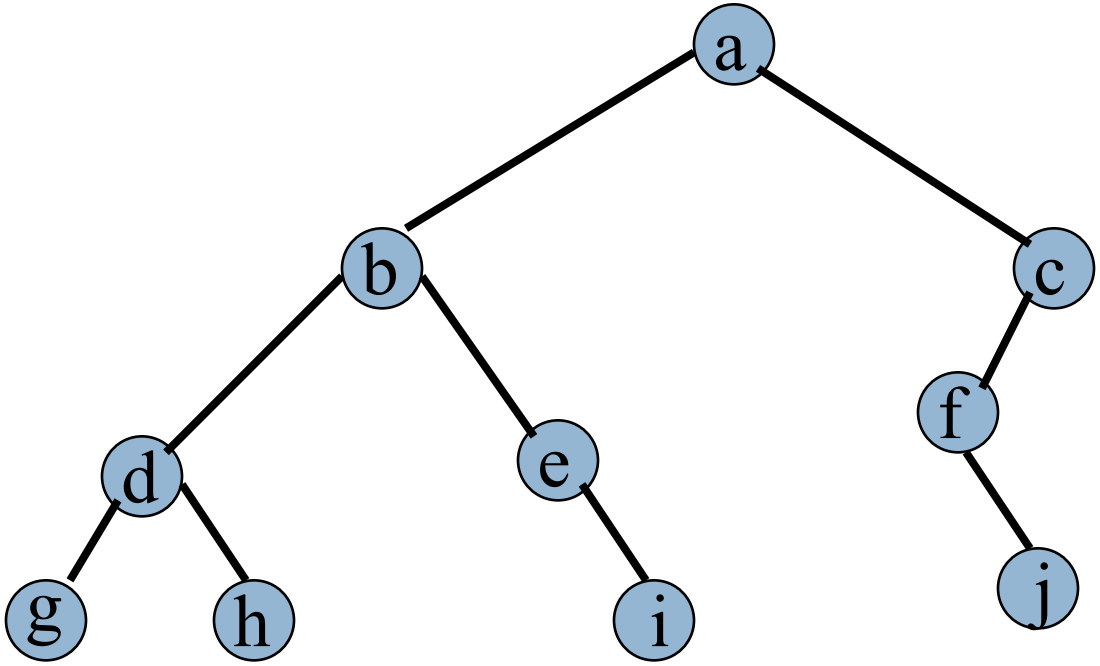


b a c

Algorithm

- Until all nodes are traversed −
  - **Step 1** − Recursively traverse left subtree.
  - **Step 2** − Visit root node.
  - **Step 3** − Recursively traverse right subtree.

Inorder Example (Visit = print)

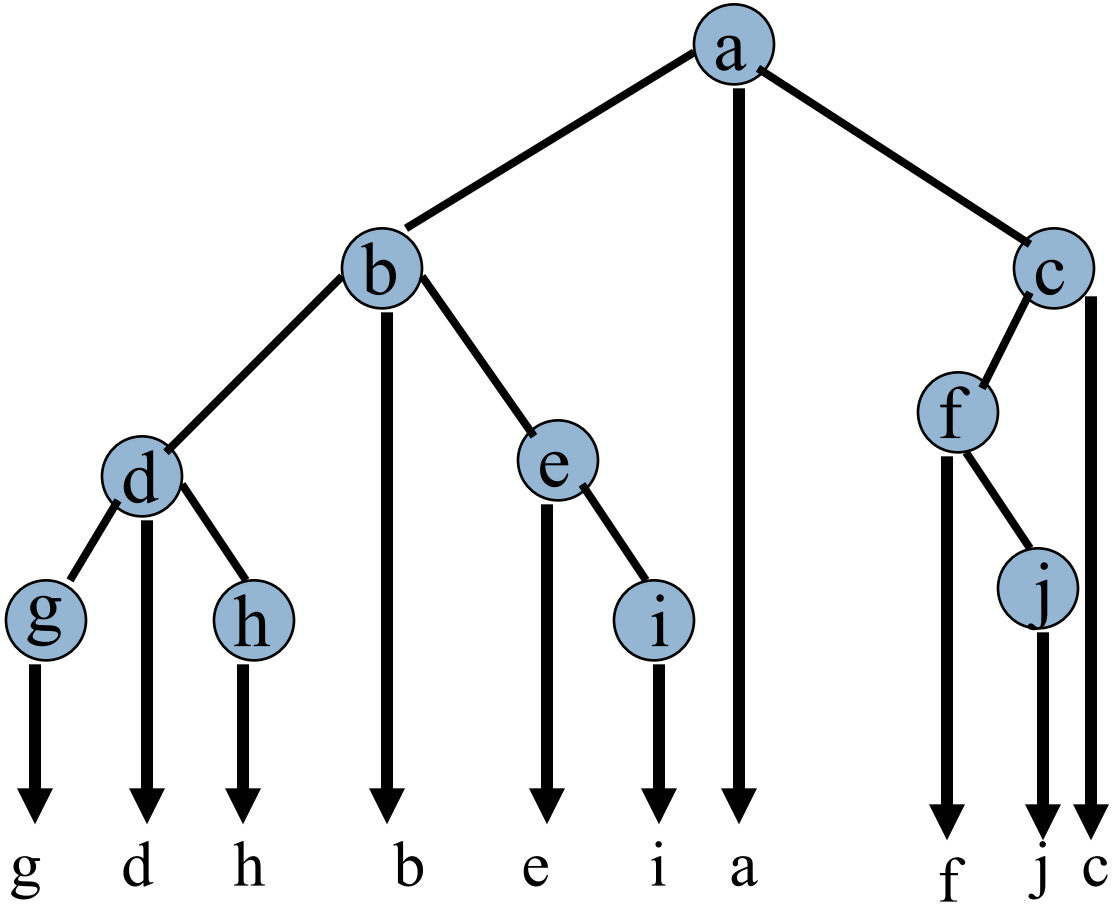


g d h b e i a f j c

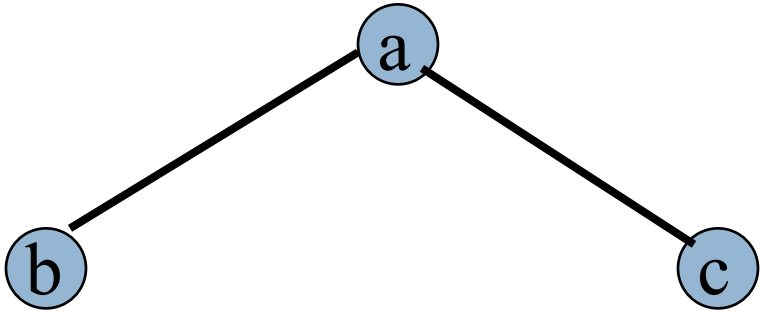
Algorithm

- Until all nodes are traversed −
  - **Step 1** − Recursively traverse left subtree.
  - **Step 2** − Visit root node.
  - **Step 3** − Recursively traverse right subtree.

Inorder By Projection (Squishing)



Postorder Example (Visit = print)

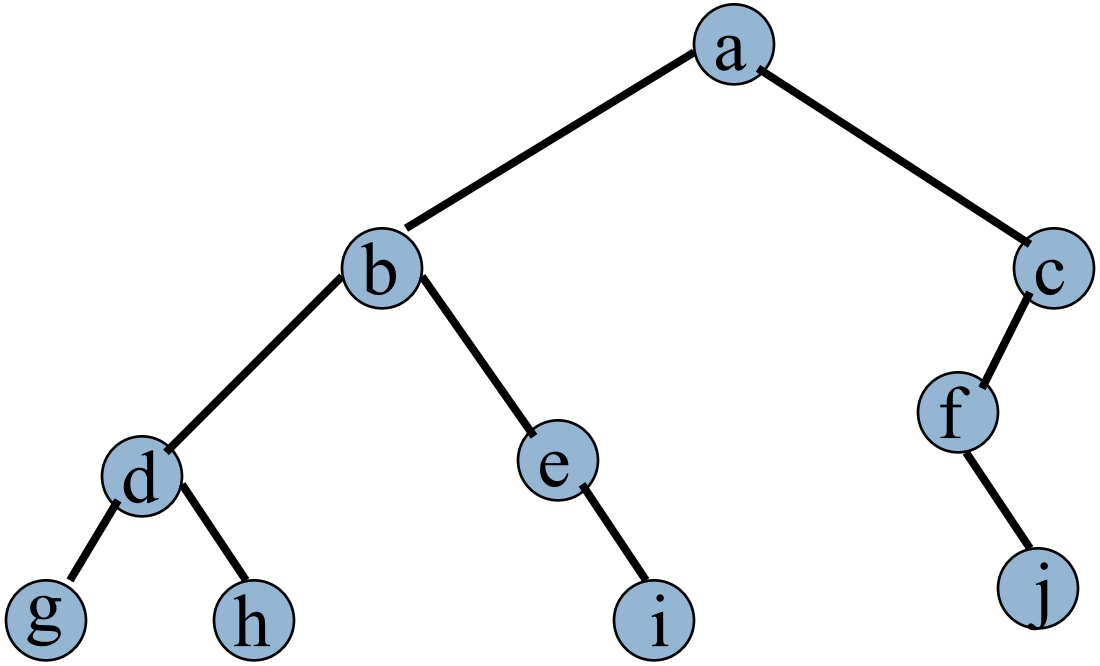


b c a

Algorithm

- Until all nodes are traversed −
  - **Step 1** − Recursively traverse left subtree.
  - **Step 2** − Recursively traverse right subtree.
  - **Step 3** − Visit root node.

Postorder Example (Visit = print)

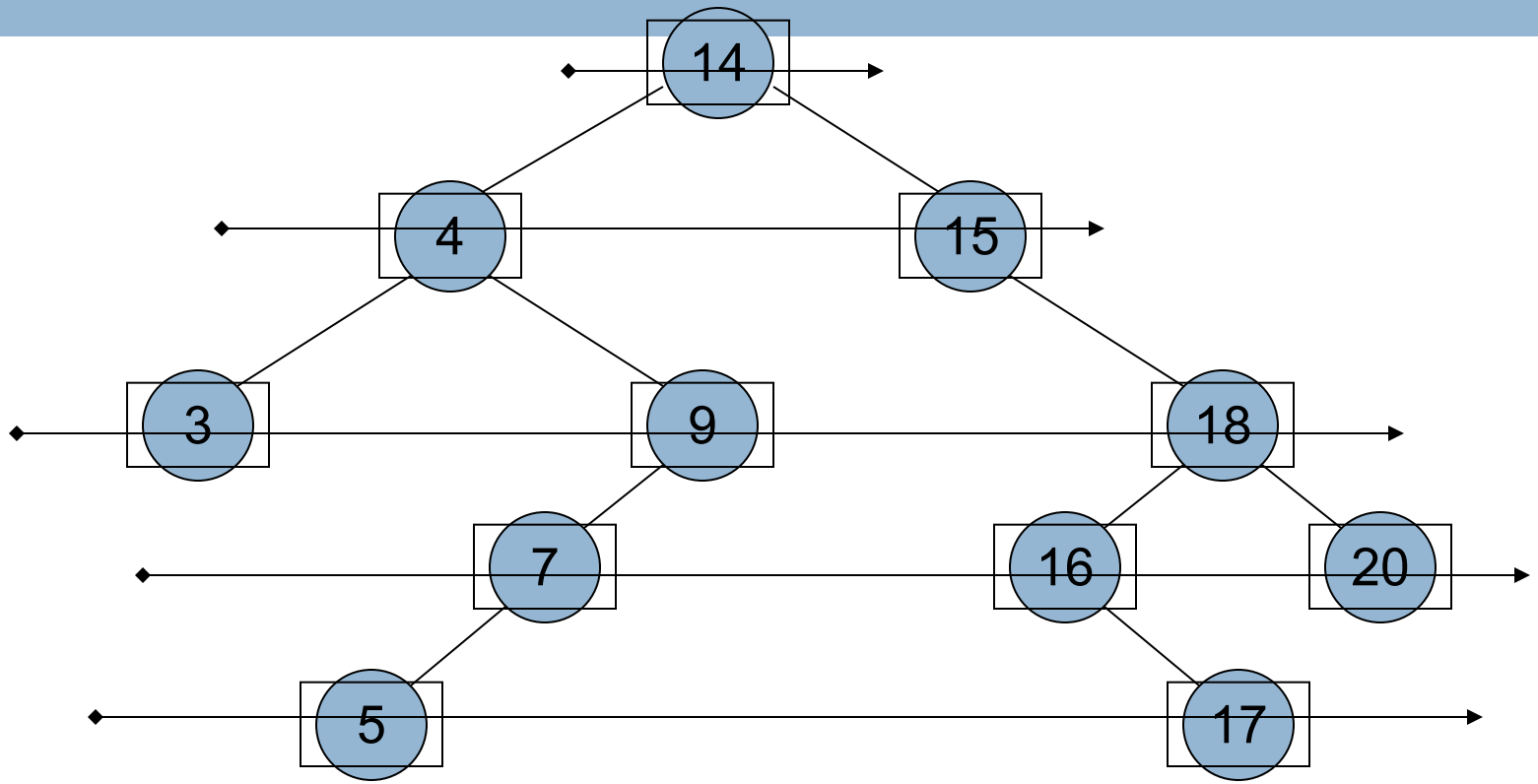


g h d i e b j f c a

Algorithm

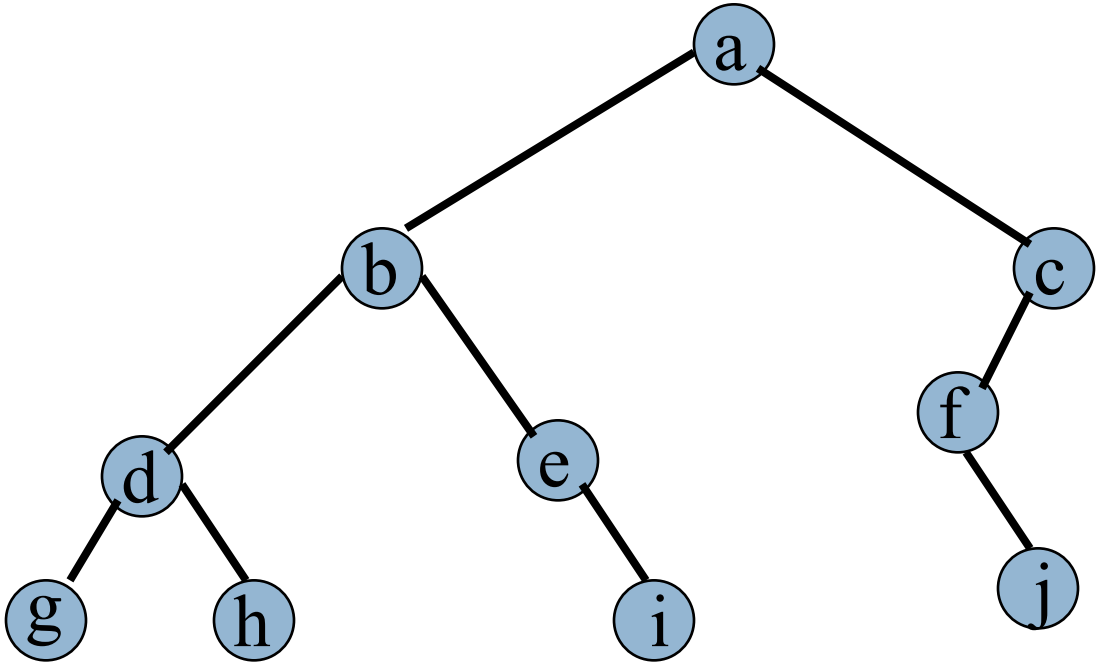
- Until all nodes are traversed −
  - **Step 1** − Recursively traverse left subtree.
  - **Step 2** − Recursively traverse right subtree.
  - **Step 3** − Visit root node.

Level-order Traversal



Level-order: 14 4 15 3 9 18 7 16 20 5 17

Level-Order Example (Visit = print)



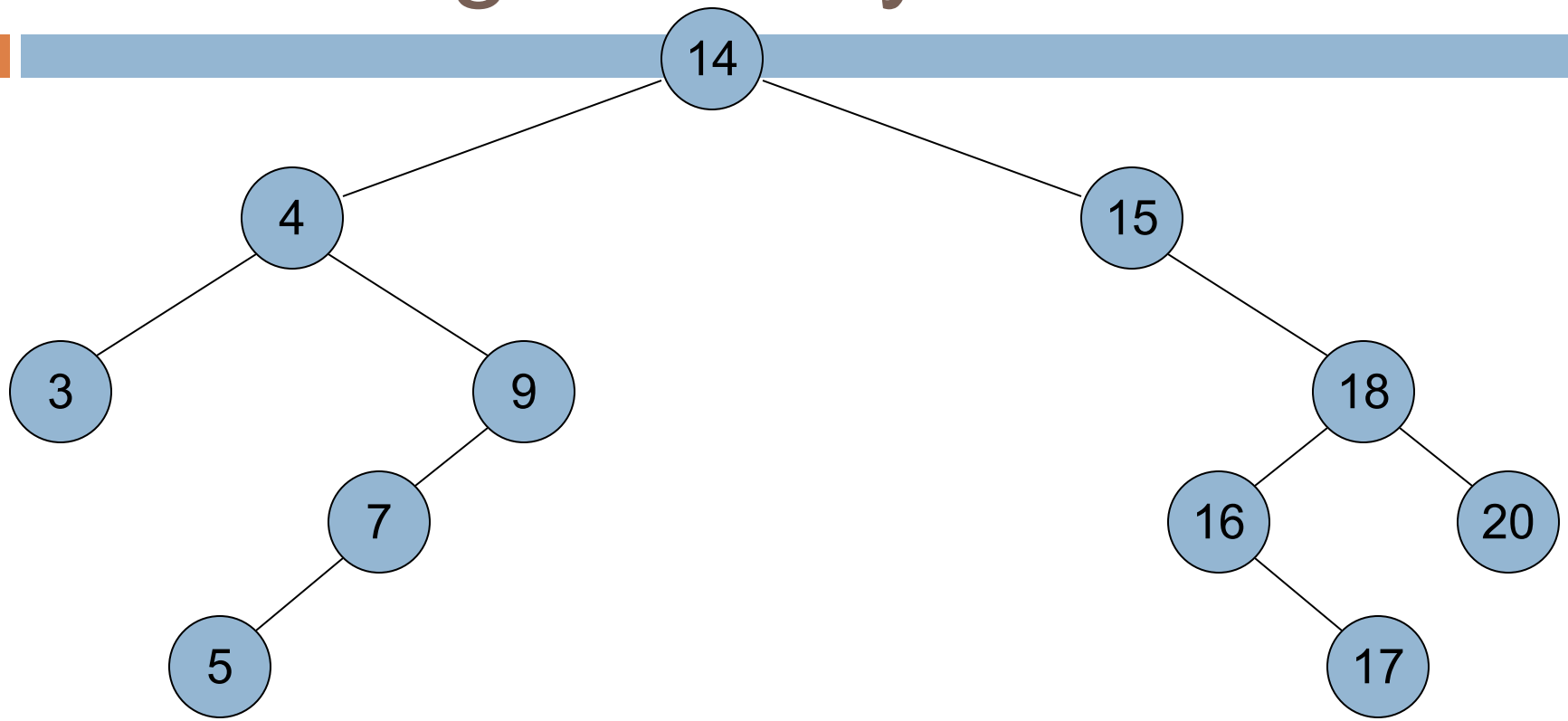
a b c d e f g h i j

Note: ***It is not recursive***

Algorithm

- Until all nodes are traversed –
  - **Step 1** – Visit each node in each level before proceeding to next level
  - **Step 2** – Visit the nodes is left to right order

Traversing a Binary Tree



Preorder: 14 4 3 9 7 5 15 18 16 17 20

Inorder: 3 4 5 7 9 14 15 16 17 18 20

Postorder: 3 5 7 9 4 17 16 20 18 15 14

Level order: 14 4 15 3 9 18 7 16 20 5 17